Assignment 3 - Graphs

Dyussenbay Damir, SE-2523

1. Implementation and complexity

The project uses Java and an adjacency list. The same Graph class stores directed and undirected graphs. The required algorithms are DSU, Kruskal, Prim, Kosaraju, Kahn and shortest paths in a DAG. Loops, parallel edges, isolated vertices and disconnected graphs are supported.

Algorithm	Time	Extra space	Explanation
DSU find / union	O(alpha(V)) amortized	O(1) per call; O(V) stored	Union by size and path compression. Creating V sets takes O(V).
Kruskal	O(V + E log(E+1))	O(V + E)	Sort edges, then use DSU to check whether an edge joins different components.
Prim	O(V + E log(E+1))	O(V + E)	The lazy priority queue may contain O(E) edges. Each adjacency entry is scanned once.
Kosaraju	O(V + E)	O(V + E)	Two DFS passes and a reversed graph. Both DFS passes use explicit stacks.
Kahn	O(V + E)	O(V)	Process zero-indegree vertices and decrease the indegrees of their neighbors.
DAG shortest paths	O(V + E)	O(V)	Get a topological order and relax outgoing edges once.

Space excludes the input graph. The lazy Prim implementation uses log E because this graph type allows parallel edges; an eager heap with one entry per vertex can use log V. DSU has an amortized bound: it does not mean that every individual call takes exactly constant time.

Why Kruskal and Prim are correct

Assume the chosen forest F is contained in a minimum spanning forest. Consider a cut that does not split any tree of F, and let e be its lightest crossing edge. If an optimum already contains e, the edge is safe. Otherwise, adding e to that optimum creates a cycle containing another crossing edge f. Since the weight of e is at most the weight of f, replacing f with e keeps a spanning forest without increasing its weight. Kruskal applies this property between DSU components, and Prim applies it between visited and unvisited vertices, so both keep choosing safe edges.

Why one DAG relaxation pass is enough

A virtual source has a zero-weight edge to every original vertex with indegree zero. Every vertex of a finite DAG is reachable from one of these sources. In a topological order, all predecessors of a vertex are processed before it. Assume their distances are already correct. Since every path into the vertex ends with an edge from a predecessor, all candidates have been considered when the vertex is reached. Its distance is therefore final, and relaxing its outgoing edges proves the same statement for later vertices.

2. Tests and benchmark results

The generator uses a new Random(42) for each graph, the shuffle and edge-generation rules from the assignment, and weights from 1 to 1000. There are five sizes and two densities. The benchmark runs each algorithm five times and records the median time. All 30 reference values matched; the main CSV contains 50 rows.

n	m	MST weight	SCC count	DAG distance sum
1000	4000	147970	27	396274
1000	20000	30363	1	252411
3162	12648	468911	137	1271911
3162	63240	97589	1	723157
10000	40000	1486139	373	3941828
10000	200000	309550	1	2381342
31623	126492	4715515	1188	12551891
31623	632460	968517	1	7376612
100000	400000	15012094	3923	40096539
100000	2000000	3056681	1	23512087

The benchmark checks equality of Kruskal and Prim, validates every topological order, and checks the reference results and exact scan counts. Nine JUnit tests passed. They include 100 random MST comparisons, 40 SCC comparisons with BFS reachability, 100 shortest-path comparisons with Bellman-Ford (including negative weights), file examples and a path of 100,000 vertices. The measured DFS stack depth on that path was 100,000, without recursion.

Counters divided by complexity formulas

Counter	Formula	Sparse range	Dense range
Kruskal comparisons	E log2 E	0.896-0.949	0.922-0.953
Prim comparisons	E log2 E	1.831-1.946	1.826-1.892
Kosaraju scans	V + E	1.600	1.905
Kahn scans	V + E	0.800	0.952
DAG relaxation scans	V + E	0.800	0.952

The traversal ratios are constant. The sorting and heap ratios remain close as n grows, which supports the expected complexity. For Kruskal, dsu_ops/E ranges from 2.290 to 2.748 on sparse graphs and from 0.565 to 0.894 on dense graphs. Early stopping explains the variation. If S edges are examined and C components remain, dsu_ops = 2S + 3(V-C), so DSU calls are O(E+V). These call counts do not measure parent-pointer steps and cannot by themselves prove the alpha(V) bound.

The run used OpenJDK 27 on Windows 11 with a 2 GiB heap. On the largest dense graphs, the median times in milliseconds were: Kruskal 1113.7, Prim 2846.4, Kosaraju 989.7, Kahn 260.4, DagShortestPath 343.4. Small timings are noisy because of JIT compilation, garbage collection and other programs. Different computers and JVMs change the times, so the conclusions use the operation counters.

3. Plots and slope check

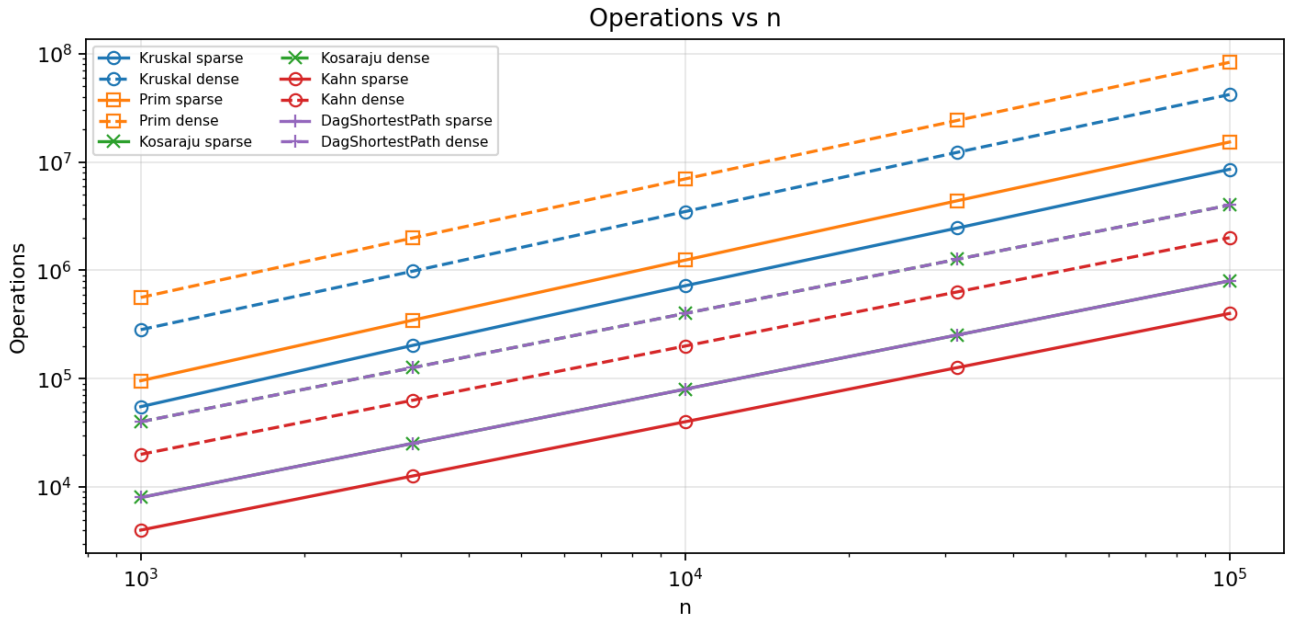


Figure 1. Operations = dsu_ops + edge_scans + comparisons. Solid and dashed lines show sparse and dense graphs. Kosaraju and DAG relaxation overlap at 2m counted operations. max_depth is a maximum stack size, so it is not added to operation counts.

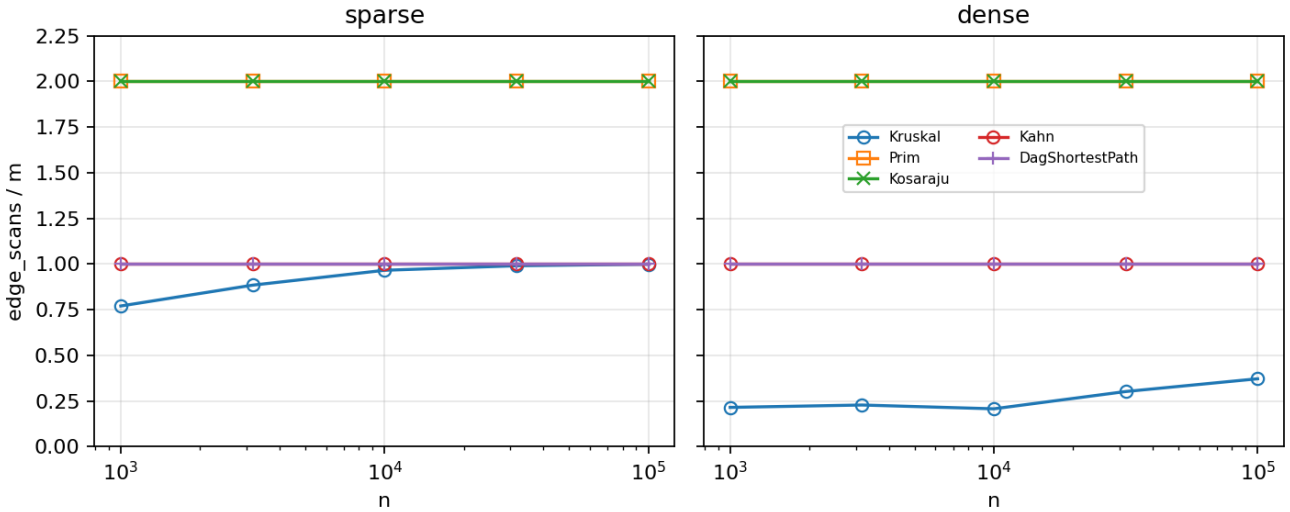


Figure 2. Kosaraju and Prim both scan 2m adjacency entries, while Kahn and DAG relaxation scan m each. These lines overlap. Kruskal may stop before examining all sorted edges.

Algorithm	Scan slope: sparse / dense	Operation slope: sparse / dense
Kruskal	1.055 / 1.119	1.093 / 1.088
Prim	1.000 / 1.000	1.102 / 1.085
Kosaraju	1.000 / 1.000	1.000 / 1.000
Kahn	1.000 / 1.000	1.000 / 1.000
DagShortestPath	1.000 / 1.000	1.000 / 1.000

Slopes are fitted by least squares to log(counter) against log(n), separately for each density. Linear traversals have slope 1.000. Kruskal sorting increments comparisons, not edge_scans, so the slightly larger operation slope shows its sorting cost. Its scan slope also depends on how early the tree finishes; scans are still bounded by E.

4. Choosing algorithms

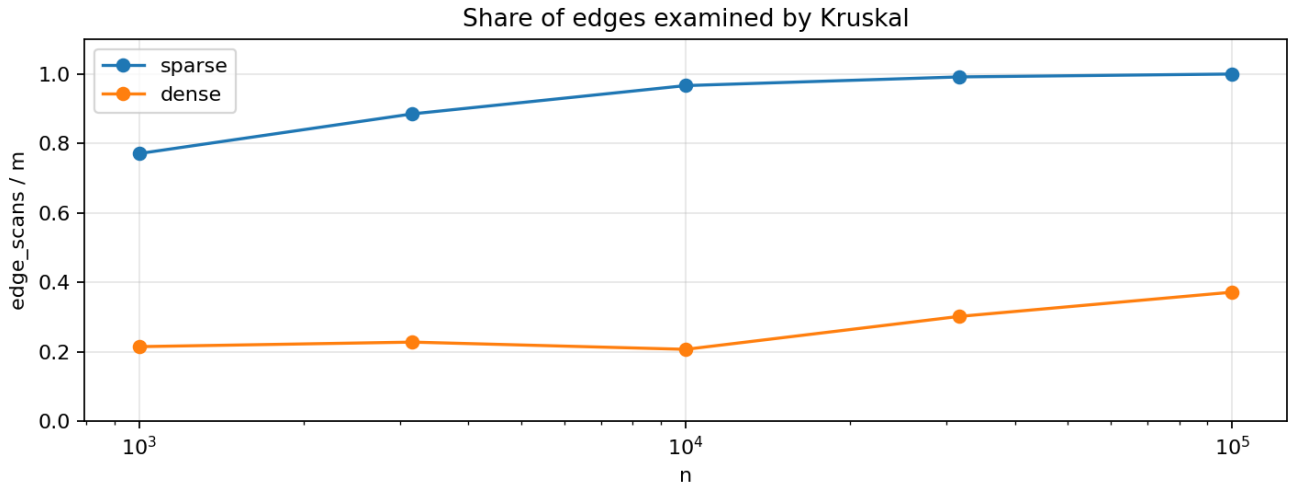


Figure 3. For n = 100,000, Kruskal examines 99.9% of the sparse edges and 37.2% of the dense edges. It still sorts every edge. On the largest dense graph, it uses 39,423,544 weight comparisons, while lazy Prim uses 79,208,272. Kruskal has less counted work in this experiment, but this does not make it the best choice for every representation.

Situation	Algorithm	Reason
Undirected edge list	Kruskal	Sorting is convenient, and DSU checks whether adding an edge forms a cycle.
Undirected adjacency lists	Prim	Grows the tree using adjacent edges. The lazy queue can take O(E) memory.
Very dense simple graph	Array-based Prim	An O(V squared) version can avoid heap overhead. This is not the version implemented here.
Directed graph with possible cycles	Kosaraju	Finds strongly connected groups, including possible deadlocks.
Need an order of dependent jobs	Kahn	Returns a valid order or reports a cycle.
Need shortest paths in a DAG	Topological relaxation	Runs in O(V+E), including when edges have negative weights.

Both assignment densities have E proportional to V: sparse is 4V and dense is 20V. The second case is denser, but it is not a quadratic-density graph. The representation and memory requirements also matter when choosing an algorithm.

Negative weights and restored paths

Negative weights are safe in a DAG because there is no directed cycle to repeat. In a general graph, a reachable negative cycle can make shortest-walk distances unbounded below. Bellman-Ford handles negative edges and detects such cycles, while Dijkstra needs nonnegative weights. The project keeps parent[] to restore paths and uses long for distances and MST totals.

What the counters include

dsu_ops counts find calls and successful unions. comparisons counts sort/heap weight comparisons or comparisons of candidate distances. Kosaraju edge_scans counts only the two DFS passes, excluding graph reversal. DAG edge_scans counts only relaxation; its Kahn preprocessing has separate counters, but is included in elapsed time. Prim starts at vertex 0 and restarts for disconnected components. Graph generation and validation are outside the measured time.

Source of the specification and reference values: the supplied Assignment 3 PDF. Optional bonus tasks are not included.